



SC2001 Lab 3

Dynamic programming

- Wang Yongqing
- Johannes Yap
- Ong Jee Shen
- Balodi Shalok

SDDB Group 6

Question 1

Recursive Definition of Function $P(C)$

Recursive Definition of Function P(C)

We're solving the unbounded knapsack problem, meaning we can take unlimited copies of each item.

Let:

- Knapsack capacity C
- n types of objects
- Each type i has weight w_i and profit p_i
- Unlimited supply of each type

We define $P(C)$ = maximum Profit for capacity C

Recursive Definition

$$P(C) = \begin{cases} 0, & \text{if } C = 0 \\ \max_{i: w_i \leq C} \{p_i + P(C - w_i)\}, & \text{if } C > 0 \end{cases}$$

Question 2

Subproblem Graph for $P(14)$

Constructing the Graph

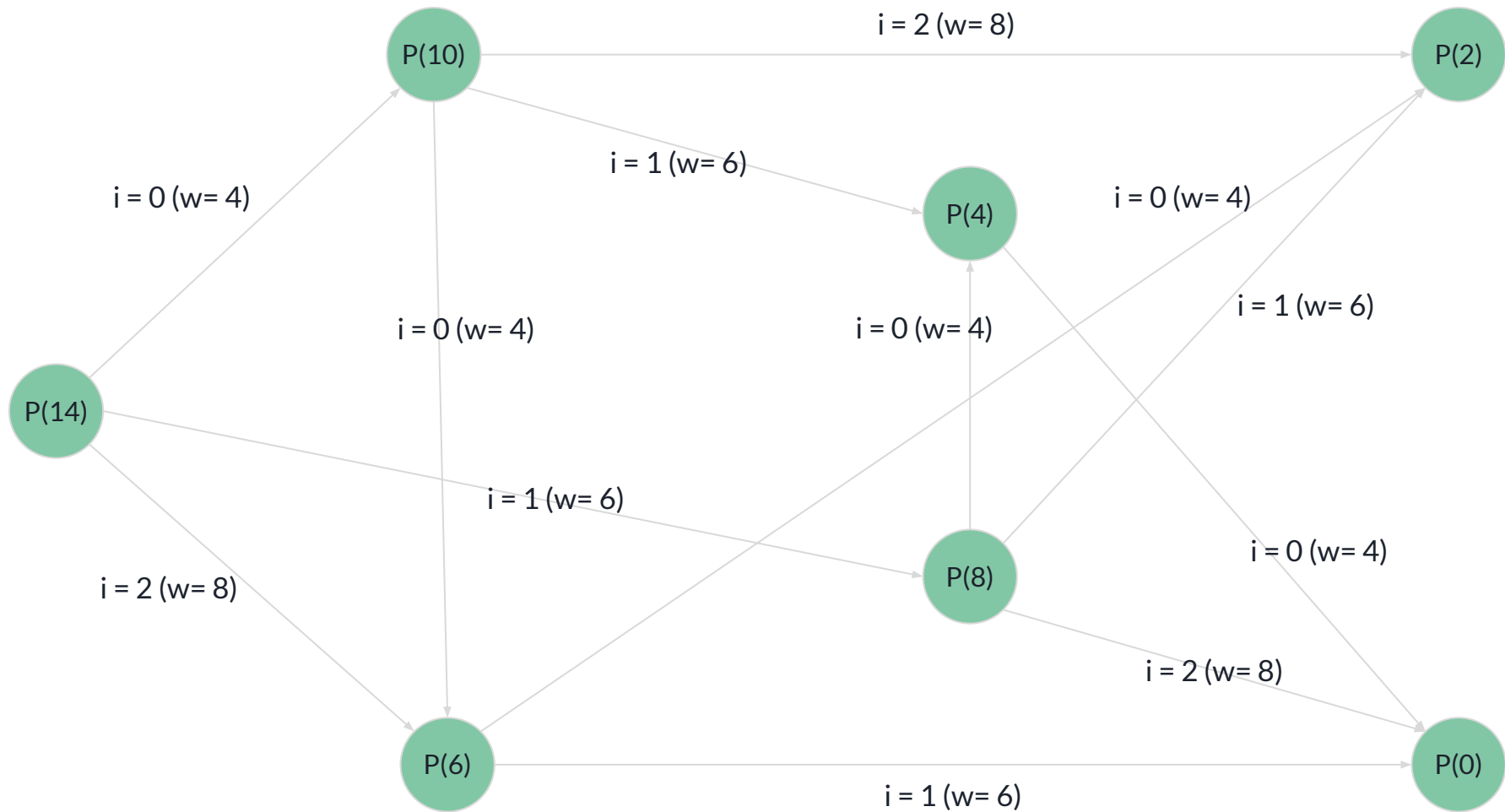
How to construct the graph

i	0	1	2
w_i	4	6	8
p_i	7	6	9

- Each node represents a subproblem $P(c)$ for some capacity c
- Directed edge goes from $P(c)$ to $P(c - w_i)$ for each object i whose weight $w_i \leq c$
- $P(c)$ depends on $P(c - w_i)$

Step-by-step Expansion

From Node	$- w_0 = 4$	$- w_1 = 6$	$- w_2 = 8$
$P(14)$	$\rightarrow P(10)$	$\rightarrow P(8)$	$\rightarrow P(6)$
$P(10)$	$\rightarrow P(6)$	$\rightarrow P(4)$	$\rightarrow P(2)$
$P(8)$	$\rightarrow P(4)$	$\rightarrow P(2)$	$\rightarrow P(0)$
$P(6)$	$\rightarrow P(2)$	$\rightarrow P(0)$	—
$P(4)$	$\rightarrow P(0)$	—	—
$P(2)$	—	—	—
$P(0)$	—	—	—



Question 3

Bottom-Up Dynamic Programming Algorithm

Problem Overview

The Problem

Given:

- Knapsack capacity C
- n types of objects
- Each type i has weight w_i and profit p_i
- Unlimited supply of each type

Find:

Largest total profit fitting in knapsack

Example Input

i	0	1	2
w_i	4	6	8
p_i	7	6	9

Capacity $C = 14$

$n = 3$ types of objects

Goal: Find $P(14) = \text{max profit}$

Core Idea – Build Up From Small Subproblems

1

Define subproblems

Let $dp[c]$ = maximum profit
achievable with knapsack capacity
 c , for $c = 0, 1, \dots, C$

2

Fill in order

Compute $dp[0], dp[1], \dots, dp[C]$ in
order. Each value only needs
previously computed results.

3

Use recurrence

$dp[c] = \max\{ p_i + dp[c - w_i] \}$ for all
items i where $w_i \leq c$. Base case:
 $dp[0] = 0$.

The Recurrence

Base case: $dp[0] = 0$ (empty knapsack has zero profit)

Recurrence: $dp[c] = \max \{ p_i + dp[c - w_i] \mid w_i \leq c, i = 0 \dots n-1 \}$ for $c = 1 \dots C$

$dp[c]$

Best profit for
capacity c

p_i

Profit from
one item i

$dp[c - w_i]$

Best profit for
remaining space

$w_i \leq c$

Item must fit
in remaining space

Answer: $dp[C]$ — the last cell gives the maximum profit for the full capacity

Algorithm – Pseudocode

```
function UnboundedKnapsack(C, w[], p[], n):
```

```
    dp[0..C] = 0
```

```
    for c = 1 to C:
```

```
        best = 0
```

```
        for i = 0 to n-1:
```

```
            if w[i] ≤ c:
```

```
                best = max(best, p[i] + dp[c - w[i]])
```

```
        dp[c] = best
```

```
    return dp[C]
```

```
// initialise all entries to 0
```

```
// iterate over every capacity
```

```
// reset best profit for this c
```

```
// try every item type
```

```
// item must fit in remaining space
```

```
// update if better profit found
```

```
// store best profit for capacity c
```

```
// answer: max profit for full capacity
```

Trace – dp[0] to dp[8] (items: w={4,6,8}, p={7,6,9})

c	Items tried ($w_i \leq c$)	Recurrence	dp[c]
0	—	base case	0
1	none (all $w_i > 1$)	dp[1] = 0	0
2	none (all $w_i > 2$)	dp[2] = 0	0
3	none (all $w_i > 3$)	dp[3] = 0	0
4	w=4 only	$p_0 + dp[0] = 7 + 0 = 7$	7
5	w=4 only	$p_0 + dp[1] = 7 + 0 = 7$	7
6	w=4, w=6	$\max(7 + dp[2], 6 + dp[0]) = \max(7, 6) = 7$	7
7	w=4, w=6	$\max(7 + dp[3], 6 + dp[1]) = \max(7, 6) = 7$	7
8	w=4, w=6, w=8	$\max(7 + dp[4], 6 + dp[2], 9 + dp[0]) = \max(14, 6, 9) = 14$	14

Highlighted row: dp[8] = 14 (two items of weight 4, profit 7 each)

Complexity Analysis

Time Complexity

$$O(n \cdot C)$$

Outer loop runs C times.

Inner loop runs n times.

Total = $n \times C$ operations.

Space Complexity

$$O(C)$$

Only a 1D array $dp[0..C]$
is needed.

No 2D table required.

vs. Recursion

No re-computation

Each subproblem is solved
exactly once.

Overlapping subproblems
are reused directly.

Summary

Bottom-Up Dynamic Programming Algorithm in 4 Steps

1

Initialise

Set $dp[0..C] = 0$

2

Iterate

For $c = 1$ to C

3

Maximise

$dp[c] = \max\{ p_i + dp[c-w_i] \}$

4

Return

Answer is $dp[C]$

Time: $O(n \cdot C)$ · Space: $O(C)$ · Each subproblem solved exactly once

C++ Code Implementation

RunCase() Function

```
void runCase(const string& label,
            int capacity,
            const vector<int>& weights,
            const vector<int>& profits) {

    cout << "-----\n";
    cout << label << "\n";
    cout << "-----\n";
    cout << " Capacity C = " << capacity << "\n";

    cout << " Weights : ";
    for (size_t i = 0; i < weights.size(); ++i)
        cout << weights[i] << (i + 1 < weights.size() ? ", " : "\n");

    cout << " Profits : ";
    for (size_t i = 0; i < profits.size(); ++i)
        cout << profits[i] << (i + 1 < profits.size() ? ", " : "\n");

    vector<int> dp = unboundedKnapsackBottomUp(capacity, weights, profits);

    // Print the full dp table
    cout << "\n Bottom-up DP table (dp[0] to dp[" << capacity << "]):\n ";
    for (int c = 0; c <= capacity; ++c)
        cout << "dp[" << c << "]=" << dp[c]
            << (c < capacity ? " " : "\n");

    cout << "\n Maximum profit P(" << capacity << ") = " << dp[capacity] << "\n";
```

```
    // Trace back which items were chosen
    cout << "\n Items chosen (greedy trace from dp table):\n";
    int rem = capacity;
    int itemCount = 0;
    while (rem > 0) {
        bool found = false;
        for (size_t i = 0; i < weights.size(); ++i) {
            if (weights[i] <= rem &&
                dp[rem] == profits[i] + dp[rem - weights[i]]) {
                cout << " Take item " << i
                    << " (w=" << weights[i]
                    << ", p=" << profits[i]
                    << "), remaining capacity: "
                    << rem << " -> " << rem - weights[i] << "\n";
                rem -= weights[i];
                ++itemCount;
                found = true;
                break;
            }
        }
        if (!found) break; // e.g. P(2) = 0, nothing fits
    }
    if (itemCount == 0)
        cout << " (no items fit - profit = 0)\n";

    cout << "\n";
}
```


printSubproblemGraph() Function

```
void printSubproblemGraph() {
    cout << "-----\n";
    cout << "(2) Subproblem Graph for P(14)\n";
    cout << "    Items: w={4,6,8}, p={7,6,9}\n";
    cout << "-----\n";
    cout << "    Reachable nodes: { 14, 10, 8, 6, 4, 2, 0 }\n\n";
    cout << "    Directed edges (P(c) --> P(c - w_i)):\n";
    cout << "        P(14) --> P(10) [w=4], P(8) [w=6], P(6) [w=8]\n";
    cout << "        P(10) --> P( 6) [w=4], P(4) [w=6], P(2) [w=8]\n";
    cout << "        P( 8) --> P( 4) [w=4], P(2) [w=6], P(0) [w=8]\n";
    cout << "        P( 6) --> P( 2) [w=4], P(0) [w=6]\n";
    cout << "        P( 4) --> P( 0) [w=4]\n";
    cout << "        P( 2) --> (no outgoing edges - no item fits)\n";
    cout << "        P( 0) --> (base case)\n";
    cout << "\n";
}
```

unboundedKnapsackBottomUp() Function

```
vector<int> unboundedKnapsackBottomUp(int capacity,
                                     const vector<int>& weights,
                                     const vector<int>& profits) {
    vector<int> dp(capacity + 1, 0);

    for (int c = 1; c <= capacity; ++c) {
        for (size_t i = 0; i < weights.size(); ++i) {
            if (weights[i] <= c) {
                dp[c] = max(dp[c], profits[i] + dp[c - weights[i]]);
            }
        }
    }
    return dp;
}
```

main() Function

```
int main() {  
    // -----  
    // (1) Recursive definition  
    // -----  
    cout << "=====\n";  
    cout << "(1) Recursive Definition of P(C)\n";  
    cout << "=====\n";  
    cout << "   Base case : P(0) = 0\n";  
    cout << "   Recursive : P(C) = max{ p_i + P(C - w_i) | w_i <= C }\n";  
    cout << "           for i = 0, 1, ..., n-1 (C > 0)\n\n";  
  
    // -----  
    // (2) Subproblem graph  
    // -----  
    printSubproblemGraph();  
  
    // -----  
    // (3) Algorithm description (printed, not just a comment)  
    // -----  
    cout << "=====\n";  
    cout << "(3) Bottom-Up Dynamic Programming Algorithm\n";  
    cout << "=====\n";  
    cout << "   1. Initialise dp[0..C] = 0.\n";  
    cout << "   2. For c = 1 to C:\n";  
    cout << "       For each item i (0 to n-1):\n";  
    cout << "           If w_i <= c:\n";  
    cout << "               dp[c] = max(dp[c], p_i + dp[c - w_i])\n";  
    cout << "   3. Return dp[C].\n";  
    cout << "\n Time: O(C * n)\n";  
    cout << " Space: O(C)\n\n";  
}
```

```
// -----  
// (4a) P(14) with weights {4,6,8}, profits {7,6,9}  
// -----  
cout << "=====\n";  
cout << "(4) Running Results\n";  
cout << "=====\n\n";  
  
runCase("(4a) weights={4,6,8}, profits={7,6,9}",  
        14, {4, 6, 8}, {7, 6, 9});  
  
// -----  
// (4b) P(14) with weights {5,6,8}, profits {7,6,9}  
// -----  
runCase("(4b) weights={5,6,8}, profits={7,6,9}",  
        14, {5, 6, 8}, {7, 6, 9});  
  
return 0;  
}
```

Question 4

**Running result of $P(14)$ with
weights and profits in cases a and b**

Case 4(a)

Output of $P(14)$ with weights and profits given below

	0	1	2
w_i	4	6	8
p_i	7	6	9

Maximum profit $P(14) = 21$

```
(4a) weights={4,6,8}, profits={7,6,9}
```

```
-----  
Capacity C = 14
```

```
Weights : 4, 6, 8
```

```
Profits : 7, 6, 9
```

```
Bottom-up DP table (dp[0] to dp[14]):
```

```
dp[0]=0 dp[1]=0 dp[2]=0 dp[3]=0 dp[4]=7 dp[5]=7 dp[6]=7  
dp[7]=7 dp[8]=14 dp[9]=14 dp[10]=14 dp[11]=14 dp[12]=21  
dp[13]=21 dp[14]=21
```

```
Maximum profit P(14) = 21
```

```
Items chosen (greedy trace from dp table):
```

```
Take item 0 (w=4, p=7), remaining capacity: 14 -> 10
```

```
Take item 0 (w=4, p=7), remaining capacity: 10 -> 6
```

```
Take item 0 (w=4, p=7), remaining capacity: 6 -> 2
```

Case 4(B)

Output of $P(14)$ with weights and profits given below

	0	1	2
w_i	5	6	8
p_i	7	6	9

Maximum profit $P(14) = 16$

4b) weights={5,6,8}, profits={7,6,9}

Capacity $C = 14$

Weights : 5, 6, 8

Profits : 7, 6, 9

Bottom-up DP table (dp[0] to dp[14]):

dp[0]=0 dp[1]=0 dp[2]=0 dp[3]=0 dp[4]=0 dp[5]=7 dp[6]=7
dp[7]=7 dp[8]=9 dp[9]=9 dp[10]=14 dp[11]=14 dp[12]=14
dp[13]=16 dp[14]=16

Maximum profit $P(14) = 16$

Items chosen (greedy trace from dp table):

Take item 0 ($w=5$, $p=7$), remaining capacity: 14 -> 9

Take item 2 ($w=8$, $p=9$), remaining capacity: 9 -> 1



Thank You